Contents

File Handling Cheat Sheet **1**
Tabla de Contenidos . . . 1
1. Lectura de Archivos . . . 1
2. Escritura de Archivos . . . 2
3. Modos de Apertura . . . 3
4. Trabajar con Paths . . . 4
5. JSON . . . 6
6. CSV . . . 7
7. Excepciones . . . 8
8. Context Managers . . . 11
9. Operaciones de Sistema . . . 13
10. Buenas Prácticas . . . 14
Recursos Relacionados . . . 17

File Handling Cheat Sheet

Python 3.13+ | Archivos, Excepciones, Context Managers

Tabla de Contenidos

1. Lectura de Archivos
2. Escritura de Archivos
3. Modos de Apertura
4. Trabajar con Paths
5. JSON
6. CSV
7. Excepciones
8. Context Managers
9. Operaciones de Sistema
10. Buenas Prácticas

---

1. Lectura de Archivos

Lectura Básica con `with`

```python
#  Forma recomendada - with cierra automáticamente
with open("archivo.txt", "r", encoding="utf-8") as file:
    content = file.read()

#  No recomendado - debe cerrar manualmente
file = open("archivo.txt", "r")
content = file.read()
file.close()
```

Métodos de Lectura

```python
with open("archivo.txt", "r", encoding="utf-8") as file:
    # Leer todo el contenido
    content = file.read()

    # Leer una línea
```

```python
    file.seek(0)  # Volver al inicio
    line = file.readline()

    # Leer todas las líneas como lista
    file.seek(0)
    lines = file.readlines()

    # Leer N caracteres
    file.seek(0)
    chunk = file.read(100)

# Iterar línea por línea (eficiente en memoria)
with open("archivo.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())

# Leer todo a lista sin \n
with open("archivo.txt", "r", encoding="utf-8") as file:
    lines = file.read().splitlines()

# Leer con enumerate
with open("archivo.txt", "r", encoding="utf-8") as file:
    for num, line in enumerate(file, start=1):
        print(f"{num}: {line.strip()}")
```

Lectura de Archivos Grandes

```python
# Procesar por chunks (archivos muy grandes)
def process_large_file(filepath: str, chunk_size: int = 8192):
    with open(filepath, "r", encoding="utf-8") as file:
        while chunk := file.read(chunk_size):
            process(chunk)

# Generador para líneas
def read_lines(filepath: str):
    with open(filepath, "r", encoding="utf-8") as file:
        for line in file:
            yield line.strip()

for line in read_lines("huge_file.txt"):
    process(line)
```

---

2. Escritura de Archivos

Escritura Básica

```python
# Escribir (sobrescribe)
with open("archivo.txt", "w", encoding="utf-8") as file:
    file.write("Línea 1\n")
    file.write("Línea 2\n")

# Escribir múltiples líneas
lines = ["Línea 1", "Línea 2", "Línea 3"]
with open("archivo.txt", "w", encoding="utf-8") as file:
```

```python
    file.writelines(line + "\n" for line in lines)

# Agregar al final (append)
with open("archivo.txt", "a", encoding="utf-8") as file:
    file.write("Nueva línea al final\n")
```

Escribir con print()

```python
with open("archivo.txt", "w", encoding="utf-8") as file:
    print("Línea 1", file=file)
    print("Línea 2", file=file)
    print("Valores:", 1, 2, 3, sep=", ", file=file)
```

Escritura Atómica (Segura)

```python
import tempfile
import os
from pathlib import Path

def atomic_write(filepath: str, content: str) -> None:
    """Escritura atómica - previene corrupción."""
    path = Path(filepath)

    # Escribir a archivo temporal
    fd, tmp_path = tempfile.mkstemp(
        dir=path.parent,
        prefix=".tmp_"
    )
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as tmp:
            tmp.write(content)
        # Renombrar es atómico en la mayoría de sistemas
        os.replace(tmp_path, filepath)
    except:
        os.unlink(tmp_path)
        raise
```

---

3. Modos de Apertura

Tabla de Modos

Modo	Descripción	Archivo existe	Archivo no existe
`"r"`	Lectura (default)	Abre	Error
`"w"`	Escritura	Sobrescribe	Crea
`"a"`	Append	Agrega al final	Crea
`"x"`	Creación exclusiva	Error	Crea
`"r+"`	Lectura + Escritura	Abre	Error
`"w+"`	Escritura + Lectura	Sobrescribe	Crea
`"a+"`	Append + Lectura	Agrega/Lee	Crea

Modificadores

Modificador	Descripción
`"b"`	Modo binario (ej: `"rb"`, `"wb"`)
`"t"`	Modo texto (default)

Ejemplos

```python
# Lectura binaria (imágenes, etc.)
with open("image.png", "rb") as file:
    data = file.read()

# Escritura binaria
with open("output.bin", "wb") as file:
    file.write(b"\x00\x01\x02\x03")

# Crear solo si no existe
try:
    with open("nuevo.txt", "x", encoding="utf-8") as file:
        file.write("Contenido inicial")
except FileExistsError:
    print("El archivo ya existe")

# Lectura y escritura
with open("archivo.txt", "r+", encoding="utf-8") as file:
    content = file.read()
    file.seek(0)
    file.write("Nuevo: " + content)
```

---

4. Trabajar con Paths

pathlib (Recomendado)

```python
from pathlib import Path

# Crear Path
p = Path("folder/subfolder/file.txt")
p = Path.home() / "Documents" / "file.txt"
p = Path.cwd() / "data"

# Propiedades
p.name              # "file.txt"
p.stem              # "file"
p.suffix            # ".txt"
p.suffixes          # [".tar", ".gz"] para "file.tar.gz"
p.parent            # Path("folder/subfolder")
p.parents           # Todos los ancestros
p.parts             # ("folder", "subfolder", "file.txt")
p.anchor            # "/" en Unix, "C:\\" en Windows

# Verificaciones
p.exists()          # ¿Existe?
p.is_file()         # ¿Es archivo?
p.is_dir()          # ¿Es directorio?
p.is_absolute()     # ¿Es ruta absoluta?
```

```
p.is_symlink()      # ¿Es enlace simbólico?

# Conversión
str(p)              # "folder/subfolder/file.txt"
p.absolute()        # Ruta absoluta
p.resolve()         # Ruta canónica (resuelve .., symlinks)
p.as_uri()          # "file:///path/to/file.txt"

# Modificar ruta
p.with_name("new.txt")      # Cambiar nombre
p.with_stem("new")          # Cambiar stem
p.with_suffix(".md")        # Cambiar extensión
```

Operaciones con pathlib

```
from pathlib import Path

# Crear directorio
Path("new_folder").mkdir(exist_ok=True)
Path("a/b/c").mkdir(parents=True, exist_ok=True)

# Listar contenido
p = Path("folder")
list(p.iterdir())           # Todos los items
list(p.glob("*.txt"))       # Archivos .txt
list(p.glob("**/*.py"))     # Recursivo
list(p.rglob("*.py"))       # Atajo para **/*.py

# Leer/Escribir (métodos directos)
p = Path("file.txt")
content = p.read_text(encoding="utf-8")
p.write_text("contenido", encoding="utf-8")
data = p.read_bytes()
p.write_bytes(b"data")

# Eliminar
p.unlink()                  # Eliminar archivo
p.unlink(missing_ok=True)   # Sin error si no existe
Path("folder").rmdir()      # Eliminar directorio vacío

# Renombrar/Mover
p.rename("nuevo_nombre.txt")
p.replace("destino.txt")    # Sobrescribe si existe

# Información
p.stat()                    # Estadísticas
p.stat().st_size            # Tamaño en bytes
p.stat().st_mtime           # Tiempo de modificación
```

os.path (Legacy)

```
import os.path

# Funciones comunes
os.path.join("folder", "file.txt")    # Unir rutas
```

```
os.path.dirname("/path/to/file.txt")  # "/path/to"
os.path.basename("/path/to/file.txt") # "file.txt"
os.path.splitext("file.txt")          # ("file", ".txt")
os.path.exists("file.txt")            # ¿Existe?
os.path.isfile("file.txt")            # ¿Es archivo?
os.path.isdir("folder")               # ¿Es directorio?
os.path.abspath("file.txt")           # Ruta absoluta
os.path.expanduser("~/file.txt")      # Expandir ~
```

---

5. JSON

Lectura y Escritura

```
import json

# Leer JSON de archivo
with open("data.json", "r", encoding="utf-8") as file:
    data = json.load(file)

# Escribir JSON a archivo
with open("data.json", "w", encoding="utf-8") as file:
    json.dump(data, file, indent=2, ensure_ascii=False)

# String a dict
data = json.loads('{"name": "Alice", "age": 30}')

# Dict a string
json_str = json.dumps({"name": "Alice", "age": 30})

# Formateo bonito
json_str = json.dumps(data, indent=2, sort_keys=True)
```

Opciones de json.dump/dumps

```
import json

data = {"nombre": "María", "edad": 30, "activo": True}

json.dumps(data,
    indent=2,               # Indentación (None = una línea)
    ensure_ascii=False,     # Permitir caracteres unicode
    sort_keys=True,         # Ordenar keys
    separators=(",", ": "), # Separadores (compacto: (",", ":"))
    default=str,            # Función para tipos no serializables
)
```

Serialización Personalizada

```
import json
from datetime import datetime
from dataclasses import dataclass, asdict

# Para datetime
class DateTimeEncoder(json.JSONEncoder):
```

```python
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()
        return super().default(obj)

json.dumps({"time": datetime.now()}, cls=DateTimeEncoder)

# Para dataclass
@dataclass
class User:
    name: str
    age: int

user = User("Alice", 30)
json.dumps(asdict(user))

# Deserialización con object_hook
def decode_user(d):
    if "name" in d and "age" in d:
        return User(**d)
    return d

json.loads('{"name": "Alice", "age": 30}', object_hook=decode_user)
```

---

6. CSV

Lectura de CSV

```python
import csv

# Leer como lista de listas
with open("data.csv", "r", encoding="utf-8") as file:
    reader = csv.reader(file)
    header = next(reader)  # Primera fila (encabezados)
    for row in reader:
        print(row)  # ['value1', 'value2', ...]

# Leer como lista de diccionarios
with open("data.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)  # {'col1': 'value1', 'col2': 'value2'}
        print(row["col1"])

# Con delimitador diferente
with open("data.tsv", "r", encoding="utf-8") as file:
    reader = csv.reader(file, delimiter="\t")
```

Escritura de CSV

```python
import csv

# Escribir lista de listas
data = [
```

```
    ["name", "age", "city"],
    ["Alice", "30", "NYC"],
    ["Bob", "25", "LA"],
]

with open("output.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data)
    # O línea por línea:
    # for row in data:
    #     writer.writerow(row)

# Escribir lista de diccionarios
data = [
    {"name": "Alice", "age": 30, "city": "NYC"},
    {"name": "Bob", "age": 25, "city": "LA"},
]

with open("output.csv", "w", encoding="utf-8", newline="") as file:
    fieldnames = ["name", "age", "city"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(data)
```

Opciones de CSV

```
import csv

# Parámetros comunes
csv.reader(file,
    delimiter=",",          # Separador de campos
    quotechar='"',          # Carácter de comillas
    escapechar="\\",        # Carácter de escape
    quoting=csv.QUOTE_MINIMAL,  # Cuándo usar comillas
)

# Constantes de quoting
csv.QUOTE_MINIMAL          # Solo cuando necesario
csv.QUOTE_ALL              # Siempre
csv.QUOTE_NONNUMERIC       # Todos excepto números
csv.QUOTE_NONE             # Nunca (usar escapechar)
```

---

7. Excepciones

Sintaxis Básica

```
try:
    # Código que puede fallar
    result = 10 / 0
except ZeroDivisionError:
    # Manejar excepción específica
    print("No se puede dividir por cero")
except (ValueError, TypeError) as e:
    # Múltiples excepciones
```

```python
    print(f"Error: {e}")
except Exception as e:
    # Cualquier otra excepción
    print(f"Error inesperado: {e}")
else:
    # Ejecuta si NO hubo excepción
    print(f"Resultado: {result}")
finally:
    # SIEMPRE ejecuta
    print("Limpieza")
```

Jerarquía de Excepciones Comunes

```
BaseException
 SystemExit
 KeyboardInterrupt
 Exception
     StopIteration
     ArithmeticError
        ZeroDivisionError
        OverflowError
     AssertionError
     AttributeError
     EOFError
     ImportError
        ModuleNotFoundError
     LookupError
        IndexError
        KeyError
     NameError
     OSError
        FileNotFoundError
        FileExistsError
        PermissionError
        IsADirectoryError
     TypeError
     ValueError
     RuntimeError
```

Excepciones de Archivo Comunes

```python
from pathlib import Path

filepath = Path("archivo.txt")

try:
    content = filepath.read_text()
except FileNotFoundError:
    print("Archivo no encontrado")
except PermissionError:
    print("Sin permisos de lectura")
except IsADirectoryError:
    print("Es un directorio, no un archivo")
except OSError as e:
    print(f"Error de sistema: {e}")
```

Lanzar Excepciones

```
# raise básico
raise ValueError("Mensaje de error")

# Re-raise
try:
    risky_operation()
except ValueError:
    log_error()
    raise  # Re-lanza la misma excepción

# raise from (encadenar excepciones)
try:
    int("abc")
except ValueError as e:
    raise RuntimeError("Conversión fallida") from e
```

Excepciones Personalizadas

```
class AppError(Exception):
    """Excepción base de la aplicación."""
    pass

class ValidationError(AppError):
    """Error de validación de datos."""

    def __init__(self, field: str, message: str):
        self.field = field
        self.message = message
        super().__init__(f"{field}: {message}")

class NotFoundError(AppError):
    """Recurso no encontrado."""

    def __init__(self, resource: str, id: str):
        self.resource = resource
        self.id = id
        super().__init__(f"{resource} with id '{id}' not found")

# Uso
def validate_age(age: int) -> None:
    if age < 0:
        raise ValidationError("age", "must be non-negative")
    if age > 150:
        raise ValidationError("age", "seems unrealistic")

try:
    validate_age(-5)
except ValidationError as e:
    print(f"Validation failed: {e.field} - {e.message}")
```

assert (Solo para Desarrollo)

```
# assert se desactiva con python -O
def divide(a: int, b: int) -> float:
```

```python
    assert b != 0, "Divisor cannot be zero"
    return a / b

# NO usar para validación en producción
#  assert user_input > 0  # Se puede desactivar
#  if user_input <= 0: raise ValueError(...)
```

---

8. Context Managers

with Statement

```python
# Garantiza limpieza (close, release, etc.)
with open("file.txt", "r") as f:
    content = f.read()
# Archivo cerrado automáticamente, incluso si hay excepción

# Múltiples context managers
with open("in.txt") as f_in, open("out.txt", "w") as f_out:
    f_out.write(f_in.read())

# Python 3.10+ - paréntesis para múltiples líneas
with (
    open("file1.txt") as f1,
    open("file2.txt") as f2,
    open("file3.txt") as f3,
):
    pass
```

Crear Context Manager con Clase

```python
class FileManager:
    def __init__(self, filename: str, mode: str = "r"):
        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        """Llamado al entrar en 'with'. Retorna recurso."""
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Llamado al salir de 'with'. Limpieza."""
        if self.file:
            self.file.close()
        # Retornar True suprime la excepción
        return False

with FileManager("test.txt", "w") as f:
    f.write("Hello!")
```

Crear Context Manager con @contextmanager

```
from contextlib import contextmanager

@contextmanager
def file_manager(filename: str, mode: str = "r"):
    """Context manager usando generador."""
    file = open(filename, mode)
    try:
        yield file  # Valor disponible como 'as'
    finally:
        file.close()

with file_manager("test.txt", "w") as f:
    f.write("Hello!")

# Ejemplo: Timer
import time

@contextmanager
def timer(label: str = ""):
    start = time.perf_counter()
    try:
        yield
    finally:
        elapsed = time.perf_counter() - start
        print(f"{label}: {elapsed:.3f}s")

with timer("Processing"):
    heavy_operation()
```

Context Managers Útiles de contextlib

```
from contextlib import (
    suppress, redirect_stdout, redirect_stderr,
    ExitStack, nullcontext
)

# suppress - ignorar excepciones específicas
with suppress(FileNotFoundError):
    Path("missing.txt").unlink()
# Sin suppress: try/except más verboso

# redirect_stdout - capturar output
import io
f = io.StringIO()
with redirect_stdout(f):
    print("Hello")
output = f.getvalue()  # "Hello\n"

# ExitStack - múltiples context managers dinámicos
with ExitStack() as stack:
    files = [
        stack.enter_context(open(f"file{i}.txt"))
        for i in range(3)
    ]
```

```
    # Todos se cerrarán al salir

# nullcontext - placeholder cuando no necesitas context manager
def process(data, cm=None):
    with (cm or nullcontext()):
        return transform(data)
```

---

9. Operaciones de Sistema

Módulo os

```
import os

# Variables de entorno
os.environ["API_KEY"]                  # Obtener (KeyError si no existe)
os.environ.get("API_KEY")              # Obtener (None si no existe)
os.environ.get("API_KEY", "default")   # Con default
os.getenv("API_KEY", "default")        # Atajo

# Directorio actual
os.getcwd()                            # Obtener
os.chdir("/path/to/dir")               # Cambiar

# Crear/Eliminar directorios
os.mkdir("new_dir")                    # Crear uno
os.makedirs("a/b/c", exist_ok=True)    # Crear recursivo
os.rmdir("empty_dir")                  # Eliminar vacío
os.removedirs("a/b/c")                 # Eliminar recursivo

# Listar directorio
os.listdir(".")                        # Lista de nombres
os.scandir(".")                        # Iterator de DirEntry

# Operaciones de archivos
os.remove("file.txt")                  # Eliminar archivo
os.rename("old.txt", "new.txt")        # Renombrar
os.replace("src.txt", "dst.txt")       # Mover/reemplazar

# Permisos
os.chmod("file.txt", 0o755)            # Cambiar permisos
os.access("file.txt", os.R_OK)         # Verificar acceso

# Información del sistema
os.name                                # "posix" o "nt"
os.sep                                 # "/" o "\\"
os.linesep                             # "\n" o "\r\n"
```

Módulo shutil

```
import shutil

# Copiar
shutil.copy("src.txt", "dst.txt")        # Copiar archivo
shutil.copy2("src.txt", "dst.txt")       # Preservar metadata
```

```python
shutil.copytree("src_dir", "dst_dir")    # Copiar directorio

# Mover
shutil.move("src.txt", "dst.txt")        # Mover archivo/dir

# Eliminar
shutil.rmtree("directory")               # Eliminar directorio recursivo

# Espacio en disco
usage = shutil.disk_usage("/")
print(f"Total: {usage.total // (1024**3)} GB")
print(f"Free: {usage.free // (1024**3)} GB")

# Crear archivo comprimido
shutil.make_archive("backup", "zip", "folder_to_compress")
shutil.make_archive("backup", "tar", "folder_to_compress")

# Extraer archivo
shutil.unpack_archive("backup.zip", "extract_to/")
```

Módulo tempfile

```python
import tempfile

# Archivo temporal (se elimina al cerrar)
with tempfile.NamedTemporaryFile(mode="w", delete=True) as tmp:
    tmp.write("contenido temporal")
    print(tmp.name)  # Ruta del archivo

# Archivo temporal persistente
with tempfile.NamedTemporaryFile(delete=False) as tmp:
    tmp.write(b"data")
    temp_path = tmp.name
# Archivo persiste, eliminar manualmente

# Directorio temporal
with tempfile.TemporaryDirectory() as tmpdir:
    print(tmpdir)  # Ruta del directorio
    # Usar el directorio
# Directorio eliminado automáticamente

# Obtener directorio temporal del sistema
tempfile.gettempdir()
```

---

10. Buenas Prácticas

Siempre Usar `with`

```python
#  Correcto
with open("file.txt", "r") as f:
    content = f.read()

#  Incorrecto
f = open("file.txt", "r")
```

```
content = f.read()
f.close()  # Puede no ejecutarse si hay excepción
```

Especificar Encoding

```
#  Correcto - explícito
with open("file.txt", "r", encoding="utf-8") as f:
    content = f.read()

#  Incorrecto - depende del sistema
with open("file.txt", "r") as f:
    content = f.read()
```

Usar pathlib

```
from pathlib import Path

#  Correcto - multiplataforma, legible
path = Path("folder") / "subfolder" / "file.txt"
if path.exists():
    content = path.read_text(encoding="utf-8")

#  Incorrecto - concatenación manual
path = "folder" + "/" + "subfolder" + "/" + "file.txt"
```

Manejo de Excepciones Específico

```
#  Correcto - excepciones específicas
try:
    content = Path("file.txt").read_text()
except FileNotFoundError:
    content = ""
except PermissionError:
    raise RuntimeError("Cannot read file: permission denied")

#  Incorrecto - captura todo
try:
    content = Path("file.txt").read_text()
except:  # Captura incluso KeyboardInterrupt
    content = ""
```

Validar Inputs

```
from pathlib import Path

def read_file(filepath: str | Path) -> str:
    path = Path(filepath)

    # Validaciones
    if not path.exists():
        raise FileNotFoundError(f"File not found: {path}")

    if not path.is_file():
        raise IsADirectoryError(f"Not a file: {path}")
```

```
    if path.suffix not in [".txt", ".md"]:
        raise ValueError(f"Unsupported file type: {path.suffix}")

    return path.read_text(encoding="utf-8")
```

Usar newline="" para CSV

```
import csv

#  Correcto - evita problemas de saltos de línea
with open("data.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(data)

#  Incorrecto - puede causar líneas en blanco extra en Windows
with open("data.csv", "w", encoding="utf-8") as f:
    writer = csv.writer(f)
```

Logging en Lugar de Print

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
logger = logging.getLogger(__name__)

def process_file(path: str) -> None:
    logger.info(f"Processing: {path}")
    try:
        # procesar...
        logger.debug("File processed successfully")
    except Exception as e:
        logger.error(f"Failed to process {path}: {e}")
        raise
```

Documentar Excepciones

```
def parse_config(path: str) -> dict:
    """
    Parse configuration file.

    Args:
        path: Path to JSON config file.

    Returns:
        Configuration dictionary.

    Raises:
        FileNotFoundError: If config file doesn't exist.
        json.JSONDecodeError: If config file is invalid JSON.
        ValueError: If config is missing required fields.
    """
    import json
```

```python
    with open(path, "r", encoding="utf-8") as f:
        config = json.load(f)

    if "version" not in config:
        raise ValueError("Config missing required field: version")

    return config
```

Recursos Relacionados

- **Anterior**: OOP Python Cheat Sheet
- **Siguiente**: Testing & Debugging Cheat Sheet
- **Documentación**: docs.python.org/3/library/io.html
- **pathlib**: docs.python.org/3/library/pathlib.html

Cheat Sheet creado para el Bootcamp Python Zero to Hero - 2026